

《数据库原理及应用教程（MySQL）》

第6章 数据表中的数据查询



01

OPTION

单关系数据查询

02

OPTION

多关系数据查询

03

OPTION

子查询

04

OPTION

集合运算查询

1. 单关系数据查询结构

- ✓ 数据查询是数据库中最常用的操作
- ✓ SQL提供SELECT语句，通过查询操作可得到所需的信息
- ✓ 关系（表）的SELECT语句的一般格式为：

投影

SELECT [ALL|DISTINCT] 〈字段名〉 [AS 别名] [{, 〈字段名〉 [AS 别名]]
FROM 〈表名或者视图名〉 [[AS] 表别名]

选取

[**WHERE** 〈检索条件〉]
[**GROUP BY** <字段名>[**HAVING** <条件表达式>]]
[**ORDER BY** <字段名>[ASC|DESC]]
[**LIMIT**子句]

2. 无条件查询

- ✓ 无条件查询是指只包含“SELECT...FROM”的查询，也称作投影查询
- ✓ 投影查询相当于关系代数中的投影运算，需要注意的是，在关系代数中，投影运算之后自动消去重复行；而SQL中必须使用关键字DISTINCT才会消去重复行

【例6-1】查询数据库“teaching”中课程表的全部内容。

在课程表c中共有 3列，字段名分别是cno（课程号）、cn（课程名）和ct（课时），本例要求查询课程表c中的全部内容，可以使用以下两种方法：

(1) SELECT后面列出表中的全部字段名

```
SELECT cno, cn, ct  
FROM c;
```

(2) 用“*”表示表中的全部字段名

```
SELECT *  
FROM c;
```

cno	cn	ct
c1	Java程序设计	40
c2	程序设计基础	48
c3	线性代数	48
c4	数据结构	64
c5	数据库系统	56
c6	数据挖掘	32
c7	高等数学	60
c8	控制理论	32

2. 无条件查询

- ✓ 可以为字段名指定别名，在查询结果中，别名会代替字段名进行显示

```
SELECT cno AS 课程号, cn AS 课程名, ct AS 课时  
FROM c;
```

课程号	课程名	课时
c1	Java程序设计	40
c2	程序设计基础	48
c3	线性代数	48
c4	数据结构	64
c5	数据库系统	56
c6	数据挖掘	32
c7	高等数学	60
c8	控制理论	32

2. 无条件查询

【例6-2】查询讲授课程的教师的教师号。

```
SELECT DISTINCT tno  
FROM tc;
```

tno
t1
t2
t3
t4
t5

✓ 上述查询使用DISTINCT去掉了查询结果中的重复元组

2. 无条件查询

【例6-3】查询前3位学生的姓名、学号和专业。

```
SELECT sn, sno, maj  
FROM s  
LIMIT 3;
```

sn	sno	maj
王彤	s1	计算机
苏乐	s2	信息
林欣怡	s3	信息

- ✓ 查询结果中字段名的顺序是由SELECT后面所列字段名的顺序决定的
 - 例如，在查询结果中，字段sn在sno之前，这和学生表s中字段的排列顺序是不同的
- ✓ LIMIT子句控制查询结果中元组的数量

3. 条件查询

- ✓ 条件查询需使用WHERE子句指定查询条件
- ✓ 查询条件中，字段名与字段名之间，或者字段名与常数之间通常使用比较运算符连接
- ✓ 常用的比较运算符如表所示

运 算 符	含 义
=、>、<、>=、<=、!=、<>	比较大小
AND(&&)、OR()、NOT(!)	多重条件
BETWEEN AND, NOT BETWEEN AND	确定范围
IN, NOT IN	确定集合
LIKE, NOT LIKE	字符匹配
IS NULL, IS NOT NULL	空值

3. 条件查询

(1) 比较大小

【例6-4】查询成绩在90分及以上的选课信息。

```
SELECT *  
FROM sc  
WHERE score>=90;
```

sno	cno	score
s1	c1	90.50
s4	c1	93.00
s7	c7	100.00
s8	c3	96.00

【例6-5】查询职称为“教授”的教师号、姓名和专业。

```
SELECT tno, tn, maj  
FROM t  
WHERE prof='教授';
```

tno	tn	maj
t1	刘杨	计算机
t4	赵礼	自动化

3. 条件查询

(2) 多重条件查询

【例6-6】查询专业是“计算机”和“数学”的学生信息。

```
SELECT *  
FROM s  
WHERE maj= '计算机' OR maj = '数学';
```

sno	sn	sex	age	maj	dept
s1	王彤	女	18	计算机	信息学院
s5	魏立	男	17	数学	理学院
s6	何欣荣	女	21	计算机	信息学院
s7	赵琳琳	女	19	数学	理学院

【例6-7】查询年龄在[30, 40]之间的教师号、姓名和职称。

```
SELECT tno AS 教师号, tn AS 姓名, prof AS 职称  
FROM t  
WHERE age >= 30 AND age <= 40;
```

教师号	姓名	职称
t1	刘杨	教授
t3	顾伟	副教授
t5	赵希希	副教授
t6	张刚	讲师

✓ 逻辑运算符“OR”，可以使用“||”代替；逻辑运算符“AND”可以使用“&&”代替

3. 条件查询

(2) 多重条件查询

【例6-8】 查询年龄不在[30, 40]之间的教师号、姓名和职称。

```
SELECT tno AS 教师号, tn AS 姓名, prof AS 职称  
FROM t  
WHERE NOT (age >= 30 AND age <= 40);
```

上述语句等价于:

```
SELECT tno AS 教师号, tn AS 姓名, prof AS 职称  
FROM t  
WHERE age < 30 OR age > 40;
```

教师号	姓名	职称
t2	石丽	讲师
t4	赵礼	教授

3. 条件查询

(2) 多重条件查询

NOT、AND、OR

高

低

(用户可以使用括号改变优先级)

【例6-9】查询讲授课程号“c1”或者“c2”且开课日期在2021年9月1日及之后的教师号、课程号和开课日期。

```
SELECT tno, cno, tcdatc
FROM tc
WHERE (cno = 'c1' OR cno = 'c2') AND tcdatc >= '2021-09-01';
```

tno	cno	tcdatc
t1	c1	2021-09-03
t1	c2	2021-09-04

3. 条件查询

(3) 确定范围

- ✓ 利用“BETWEEN AND”或者“NOT BETWEEN AND”操作可以查询字段值属于或者不属于指定连续取值区间的元组

【例6-10】查询课时在[30, 40]之间的课程号、课程名和课时。

```
SELECT cno, cn, ct
FROM c
WHERE ct BETWEEN 30 AND 40;
```

上述语句等价于：

```
SELECT cno, cn, ct
FROM c
WHERE ct >= 30 AND ct <= 40;
```

cno	cn	ct
c1	Java 程序设计	40
c6	数据挖掘	32
c8	控制理论	32

3. 条件查询

(3) 确定范围

【例6-11】 查询课时不在[30, 40]之间的课程号、课程名和课时。

```
SELECT *  
FROM c  
WHERE ct NOT BETWEEN 30 AND 40;
```

上述语句等价于：

```
SELECT *  
FROM c  
WHERE ct < 30 OR ct > 40;
```

cno	cn	ct
c2	程序设计基础	48
c3	线性代数	48
c4	数据结构	64
c5	数据库系统	56
c7	高等数学	60

3. 条件查询

(4) 确定集合

✓ 利用 “IN” 或者 “NOT IN” 操作可以查询字段值属于或者不属于指定集合的元组

【例6-12】查询课程号为“c4”和“c6”的选课信息，包括学号、课程号和成绩。

```
SELECT sno, cno, score
FROM sc
WHERE cno IN ('c4', 'c6');
```

上述语句等价于：

```
SELECT sno, cno, score
FROM sc
WHERE cno = 'c4' OR cno = 'c6';
```

sno	cno	score
s2	c4	70.00
s3	c4	85.00
s2	c6	81.50
s4	c6	NULL

3. 条件查询

(4) 确定集合

- ✓ 利用 “IN” 或者 “NOT IN” 操作可以查询字段值属于或者不属于指定集合的元组

【例6-13】查询除课程号“c4”和“c6”之外其他课程的选课信息，包括学号、课程号和成绩。

```
SELECT sno, cno, score
FROM sc
WHERE cno NOT IN ('c4', 'c6');
```

上述语句等价于：

```
SELECT sno, cno, score
FROM sc
WHERE cno <> 'c4' AND cno <> 'c6';
```

- ✓ 本例查询结果中不包括cno等于“c4”和“c6”的选课记录，即从选课表sc中去掉例6-12的查询结果。

3. 条件查询

(5) 部分匹配查询

- ✓ 查询时, 如果不知道完全精确的值, 可以使用LIKE或NOT LIKE进行部分匹配查询 (也称模糊查询)
- ✓ LIKE定义的一般格式为:

<字段名> LIKE <字符串常量>

其中, 字段名必须为字符型, 字符串常量中的字符可以包含通配符, 利用这些通配符, 可以进行模糊查询, 字符串中的通配符及其功能如表所示

通 配 符	功 能	实 例
%	代表 0 个或多个字符	'ab%', 'ab'后可接任意字符串
_ (下划线)	代表一个字符	'a_b', 'a'与'b'之间可有一个字符
[]	表示在某一范围的字符	[0-9], 0~9 之间的字符
[^]	表示不在某一范围的字符	[^0-9], 不在 0~9 之间的字符

3. 条件查询

(5) 部分匹配查询

【例6-14】查询课程名中包含“程序”的课程号、课程名和课时。

```
SELECT cno AS 课程号, cn AS 课程名, ct AS 课时  
FROM c  
WHERE cn LIKE '%程序%';
```

课程号	课程名	课时
c1	Java 程序设计	40
c2	程序设计基础	48

【例6-15】查询课程名以“程序”开头的课程号、课程名和课时。

```
SELECT cno AS 课程号, cn AS 课程名, ct AS 课时  
FROM c  
WHERE cn LIKE '程序%';
```

课程号	课程名	课时
c2	程序设计基础	48

3. 条件查询

(5) 部分匹配查询

【例6-16】查询课程名不是以“数据”开头的课程信息。

```
SELECT *  
FROM c  
WHERE cn NOT LIKE '数据%';
```

【例6-17】查询课程名中第二个字符是“据”的课程信息。

```
SELECT *  
FROM c  
WHERE cn LIKE '_据%';
```

cno	cn	ct
c1	Java 程序设计	40
c2	程序设计基础	48
c3	线性代数	48
c7	高等数学	60
c8	控制理论	32

cno	cn	ct
c4	数据结构	64
c5	数据库系统	56
c6	数据挖掘	32

3. 条件查询

(6) 空值查询

- ✓ 某个字段没有值称为具有空值 (NULL)
- ✓ 通常没有为一个元组的某个字段输入值时, 该字段的值就是空值
- ✓ 空值不同于零和空格, 它不占任何存储空间
 - 例如, 某些学生选修了课程但没有参加考试, 就会造成数据表中有选课记录, 但没有考试成绩
 - 考试成绩为空值与考试成绩为0分是不同的

3. 条件查询

(6) 空值查询

【例6-18】 查询没有考试成绩的选课信息，要求显示学号和课程号。

```
SELECT sno, cno  
FROM SC  
WHERE score IS NULL;
```

sno	cno
s2	c7
s4	c6

【例6-19】 查询有考试成绩选课信息，要求显示学号和课程号。

```
SELECT sno, cno  
FROM SC  
WHERE score IS NOT NULL;
```

- ✓ 本例的查询结果是首先对选课表sc的前两列sno和cno进行投影，之后，从中去掉例6-18中的查询结果。

4. 聚合函数查询

- ✓ SQL提供了许多实用的聚合函数，增强了基本查询能力
- ✓ 常用的聚合函数如表所示

函 数 名 称	功 能
AVG	按列计算平均值
SUM	按列计算值的总和
MAX	求一列中的最大值
MIN	求一列中的最小值
COUNT	按列值统计个数

4. 聚合函数查询

【例6-20】查询学号为“s2”的学生的总分和平均分。

```
SELECT SUM(score), AVG(score)
FROM SC
WHERE sno = 's2';
```

SUM(score)	AVG(score)
208.50	69.500000

- ✓ 从上述查询结果可以看出，在使用聚合函数进行查询时，查询结果中的字段名是聚合函数的函数名
- ✓ 如果要更清楚地表示查询内容的含义，可以为聚合函数指定别名
- ✓ 例如：

```
SELECT SUM(score) AS 总分, AVG(score) AS 平均分
FROM SC
WHERE sno = 's2';
```

总分	平均分
208.50	69.500000

- ✓ 请注意，学号为“s2”的学生共选修了4门课程，但是其中1门课程的成绩为空值，因此，聚合函数SUM和AVG对其成绩进行计算时，只考虑了有效成绩，没有将空值计算在内。

4. 聚合函数查询

【例6-21】 查询课程的最高课时、最低课时和最大课时之差。

```
SELECT MAX(ct) AS 最高课时, MIN(ct) AS 最低课时,  
MAX(ct)-MIN(ct) AS 最大课时差  
FROM c;
```

最高课时	最低课时	最大课时差
64	32	32

【例6-22】 查询学号为“s1”的学生的选课门数。

```
SELECT sno, COUNT(cno) AS 选课门数  
FROM sc  
WHERE sno = 's1';
```

sno	选课门数
s1	2

- ✓ 例6-22中的COUNT(cno)也可以写为COUNT(sno)、COUNT(score)或者COUNT(*)
- ✓ 但是，如果查询学号为“s2”或者“s4”的学生的选课门数时，由于其选课成绩包含空值，则不能使用COUNT(score)，因为COUNT只对有效成绩进行计数，导致选课门数出现错误

4. 聚合函数查询

【例6-23】 查询学生表s中的专业数量。

```
SELECT COUNT(DISTINCT maj) AS 专业数量
FROM s;
```

专业数量
4

✓ 上述语句中的关键字DISTINCT不能省略，它的作用是消除重复的元组

【例6-24】 查询“信息学院”的教师数量。

```
SELECT dept, COUNT(*) AS 教师数量
FROM t
WHERE dept = '信息学院';
```

dept	教师数量
信息学院	3

- ✓ 上述语句中的COUNT(*)用来统计元组的数量，不消除重复元组，不允许使用DISTINCT关键字
- ✓ 此外，上述代码中的“*”可以用教师表中的任一字段名进行替换。

5. 分组查询

- ✓ GROUP BY子句可以将查询结果按字段列或字段列的组合在行的方向上进行分组，每组在字段列或字段列的组合上具有相同的值

【例6-25】查询选课表sc中每门课程的课程号及其选课人数。

```
SELECT cno AS 课程号, COUNT(*) AS 选课人数  
FROM sc  
GROUP BY cno;
```

课程号	选课人数
c1	3
c2	5
c3	2
c4	2
c5	2
c6	2
c7	4

5. 分组查询

【例6-26】 查询选修三门以上（含三门）课程的学生的学号和选课门数。

```
SELECT sno AS 学号, COUNT(*) AS 选课门数
FROM sc
GROUP BY sno
HAVING COUNT(*) >= 3;
```

学号	选课门数
s2	4
s3	3
s4	4
s7	3

- ✓ GROUP BY子句按学号字段sno的值分组，所有具有相同学号的元组为一组，对每一组使用函数COUNT进行计算，统计出每个学生的选课门数，HAVING子句去掉不满足COUNT(*)>=3的组。
- ✓ 当在一个SQL查询中同时使用WHERE子句、GROUP BY子句和HAVING子句时，其顺序是WHERE、GROUP BY和HAVING
- ✓ WHERE与HAVING子句的根本区别在于作用对象不同，WHERE子句作用于基本表或视图，从中选择满足条件的元组；HAVING子句作用于组，选择满足条件的组，必须用在GROUP BY子句之后，但GROUP BY子句可以没有HAVING子句

6. 查询结果排序

- ✓ 当需要对查询结果排序时，应该使用ORDER BY子句
- ✓ ORDER BY子句必须出现在其他子句之后
- ✓ 排序方式可以指定，DESC为降序，ASC为升序，缺省时为升序

【例6-27】查询学号为“s1”的学生的选课信息，要求显示学号、课程号和成绩，并且按照成绩的降序排列。

```
SELECT sno, cno, score
FROM sc
WHERE sno = 's2'
ORDER BY score DESC;
```

sno	cno	score
s2	c6	81.50
s2	c4	70.00
s2	c5	57.00
s2	c7	NULL

6. 查询结果排序

【例6-28】 查询课程信息，并且按照课时的降序排列。

```
SELECT *  
FROM c  
ORDER BY ct DESC;
```

cno	cn	ct
c4	数据结构	64
c7	高等数学	60
c5	数据库系统	56
c2	程序设计基础	48
c3	线性代数	48
c1	Java 程序设计	40
c6	数据挖掘	32
c8	控制理论	32

- ✓ 可以看出，本例的查询结果中，课程信息按照课时“ct”的降序进行排列
- ✓ 对于相同课时的课程，默认按照课程号“cno”的升序进行排列

6. 查询结果排序

【例6-29】查询课程信息，按照课时的降序排列，课时相同再按照课程名降序排列。

```
SELECT *  
FROM c  
ORDER BY ct DESC, cn DESC;
```

cno	cn	ct
c4	数据结构	64
c7	高等数学	60
c5	数据库系统	56
c3	线性代数	48
c2	程序设计基础	48
c1	Java 程序设计	40
c6	数据挖掘	32
c8	控制理论	32

- ✓ 可以看出，课时“ct”是主排序字段，课程名“cn”是次排序字段
- ✓ 首先按照课时降序排列，课时相同的课程再按照课程名的字典序降序排列

7. 限制查询结果数量

- ✓ LIMIT子句用来限制查询结果的元组数量，其语法格式为：

LIMIT [OFFSET,] row_count | row_count OFFSET offset

- ✓ OFFSET是非负整型常量，用于指定查询结果的第一行的偏移量，默认为0，表示查询结果的第1行，以此类推，OFFSET的值为1时，表示查询结果的第2行
- ✓ row_count是非负整型常量，用来指定查询结果的行数，如果row_count的值大于实际查询结果的行数，则返回实际行数
- ✓ row_count OFFSET后面的offset也是非负整型常量，row_count OFFSET offset表示查询结果从offset+1行开始，返回row_count行

7. 限制查询结果数量

【例6-30】 查询从第2位教师开始的3位教师的教师号、姓名和职称。

```
SELECT tno, tn, prof  
FROM t  
LIMIT 1, 3;
```

上述语句等价于:

```
SELECT tno, tn, prof  
FROM t  
LIMIT 3 OFFSET 1;
```

tno	tn	prof
t2	石丽	讲师
t3	顾伟	副教授
t4	赵礼	教授

7. 限制查询结果数量

【例6-31】查询选课表sc中每门课程的课程号及其选课人数，按照选课人数降序排列，并且显示前3行。

```
SELECT cno AS 课程号, COUNT(*) AS 选课人数
FROM sc
GROUP BY cno
ORDER BY 选课人数 DESC
LIMIT 3;
```

课程号	选课人数
c2	5
c7	4
c1	3

✓ 上述语句中，LIMIT中的“3”，也可以写作“0, 3”，或者“3 OFFSET 0”

01

OPTION

单关系数据查询

02

OPTION

多关系数据查询

03

OPTION

子查询

04

OPTION

集合运算查询

1. 多关系查询结构

✓ 表的连接方法有以下两种

(1) 表之间满足一定条件的行进行连接时，FROM子句指明进行连接的表名，WHERE子句指明连接的列名及其连接条件。

```
SELECT [ALL|DISTINCT][TOP N [PERCENT][WITH TIES]] <字段名> [AS 别名1] [{, <字段名> [AS 别名2]]  
FROM <表名1> [[AS] 表1别名] [{, <表名2> [[AS] 表2别名,...]]  
[WHERE <检索条件> ]  
[GROUP BY <列名1>[HAVING <条件表达式>]]  
[ORDER BY <列名2>[ASC|DESC]]
```

1. 多关系查询结构

(2) 利用关键字JOIN进行连接。

- ✓ INNER JOIN (内连接) 显示符合条件的记录, 此为默认值。
- ✓ LEFT (OUTER) JOIN称为左 (外) 连接, 用于显示符合条件的数据行以及左边表中不符合条件的数据行。此时右边数据行会以NULL来显示
- ✓ RIGHT (OUTER) JOIN称为右 (外) 连接, 用于显示符合条件的数据行以及右边表中不符合条件的数据行。此时左边数据行会以NULL来显示
- ✓ FULL (OUTER) JOIN显示符合条件的数据行以及左边表和右边表中不符合条件的数据行。此时缺乏数据的数据行会以NULL来显示
- ✓ CROSS JOIN将一个表的每一个记录和另一表的每个记录匹配成新的数据行。
- ✓ 当将JOIN关键词放于FROM子句中时, 应有关键词ON与之对应, 以表明连接的条件。

SELECT [ALL|DISTINCT][TOP N [PERCENT][WITH TIES]] 字段名1 [AS 别名1] [, 字段名2 [AS 别名2]...]

FROM 表名 1[[AS] 表1别名] [INNER|RIGHT|FULL|OUTER|CROSS] JOIN表名2 [[AS] 表2别名]

ON 条件

2. 内连接查询

【例6-32】查询学号为“s5”的学生的选课信息，要求列出学号、姓名和课程号。

(1) 方法1

```
SELECT s.sno, sn, cno
FROM s, sc
WHERE s.sno = 's5' AND s.sno = sc.sno;
```

(2) 方法2

```
SELECT s.sno, sn, cno
FROM s INNER JOIN sc
ON s.sno = sc.sno AND s.sno = 's5';
```

sno	sn	cno
s5	魏立	c2
s5	魏立	c7

- ✓ 上述语句中的学号“sno”需要加上表名前缀，这是因为学生表s和选课表sc中都有学号“sno”，必须用表名前缀来确切说明该字段属于哪个表，以避免二义性，此外，上述语句中的学号“sno”的表前缀也可以写为选课表sc
- ✓ 上述语句的执行过程是将学生表s中的学号“sno”和选课表sc中的学号“sno”进行等值连接，同时选取学号为“s5”的行，然后对学号“sno”、学生姓名“sn”和课程号“cno”进行投影操作，即可得到查询结果

2. 内连接查询

【例6-33】查询所有授课教师的教师号、姓名和讲授的课程名，并且按照教师号升序排列。

(1) 方法1

```
SELECT t.tno, tn, cn
FROM t, tc, c
WHERE t.tno = tc.tno AND tc.cno = c.cno
ORDER BY tno;
```

(2) 方法2

```
SELECT t.tno, tn, cn
FROM t INNER JOIN tc INNER JOIN c
ON t.tno = tc.tno AND tc.cno = c.cno
ORDER BY tno;
```

tno	tn	cn
t1	刘杨	Java 程序设计
t1	刘杨	程序设计基础
t2	石丽	数据库系统
t2	石丽	数据挖掘
t3	顾伟	程序设计基础
t3	顾伟	数据结构
t4	赵礼	线性代数
t5	赵希希	高等数学
t5	赵希希	控制理论

2. 内连接查询

【例6-34】查询选课人数在3人及以上的课程号、课程名和选课人数。

(1) 方法1

```
SELECT c.cno, cn, COUNT(sc.sno) as 选课人数
FROM c, sc
WHERE c.cno = sc.cno
GROUP BY c.cno, cn
HAVING 选课人数>=3;
```

(2) 方法2

```
SELECT c.cno, cn, COUNT(sc.sno) as 选课人数
FROM c INNER JOIN sc
ON c.cno = sc.cno
GROUP BY c.cno, cn
HAVING COUNT(sc.sno)>=3;
```

cno	cn	选课人数
c1	Java 程序设计	3
c2	程序设计基础	5
c7	高等数学	4

3. 外连接查询

- ✓ 在内连接查询中，不满足连接条件的元组不能作为查询结果输出
 - 例如，例6-33的查询结果只包括有授课记录的教师信息，而不会有教师号为“t6”的张刚老师信息
- ✓ 在外部连接中，参与连接的表有主从之分，以主表的每行数据去匹配从表的数据列，符合连接条件的数据将直接返回到结果集中；对那些不符合连接条件的列，将被填上NULL值后，再返回到结果集中
- ✓ 外部连接分为左外部连接和右外部连接两种
 - 主表在左边，则称为左外部连接
 - 主表在右边，则称为右外部连接

3. 外连接查询

【例6-35】查询所有教师的教师号、姓名和授课程名，并且按照教师号升序排列（没有授课的教师的授课信息显示为空）。

```
SELECT t.tno, tn, cn
FROM t LEFT OUTER JOIN tc ON t.tno = tc.tno
      LEFT OUTER JOIN c ON tc.cno = c.cno
ORDER BY tno;
```

tno	tn	cn
t1	刘杨	Java 程序设计
t1	刘杨	程序设计基础
t2	石丽	数据库系统
t2	石丽	数据挖掘
t3	顾伟	程序设计基础
t3	顾伟	数据结构
t4	赵礼	线性代数
t5	赵希希	高等数学
t5	赵希希	控制理论
t6	张刚	NULL

- ✓ 可以看出，与例6-33的查询结果相比，本例的查询结果包括所有的教师，没有授课的张刚老师的授课名称显示为空

3. 外连接查询

【例6-36】查询所有学生的学号、姓名、课程号和成绩（没有选课的学生选课信息显示为空）。

```
SELECT s.sno, sn, cno, score
FROM s LEFT OUTER JOIN sc
ON sc.sno = s.sno;
```

上述代码用了左连接，其中主表是学生表s，从表是选课表sc。本例中的查询任务也可以用右连接，如下：

```
SELECT s.sno, sn, cno, score
FROM sc RIGHT OUTER JOIN s
ON sc.sno = s.sno;
```

- ✓ 查询结果中，除了显示选课学生的信息之外，也显示未选课的学生学号为“s6”的学生信息，其选课课程号和成绩为空

4. 交叉连接查询

- ✓ 交叉连接查询 (CROSS JOIN) 对连接查询的表没有特殊的要求, 任何表都可以进行交叉连接查询操作

【例6-37】对教师表和课程表进行交叉连接查询。

```
SELECT *  
FROM t CROSS JOIN c;
```

- ✓ 上述语句是将教师表t中的每一个元组和课程表c的每一个元组匹配生成新的数据行, 查询结果的行数是两个表行数的乘积, 列数是两个表列数的和

5. 自连接查询

✓ 当一个表与其自身进行连接查询操作时，称为表的自连接查询

【例6-38】查询比“程序设计基础”课时高的课程号、课程名和课时。

要查询的内容均在同一个课程表c中，可以为课程表c分别取两个别名，一个是x，一个是y。将y中满足比“程序设计基础”课时高的行与x中的“程序设计基础”课程行连接起来，这实际上是同一课程表c的大于连接。

(1) 方法1

```
SELECT x.cno AS 课程号, x.cn AS 课程名, x.ct AS 课时
FROM c AS x, c AS y
WHERE x.ct > y.ct AND y.cn='程序设计基础';
```

(2) 方法2

```
SELECT x.cno AS 课程号, x.cn AS 课程名, x.ct AS 课时
FROM c AS x INNER JOIN c AS y
ON x.ct > y.ct AND y.cn='程序设计基础';
```

课程号	课程名	课时
c4	数据结构	64
c5	数据库系统	56
c7	高等数学	60

5. 自连接查询

【例6-39】查询与学生“王彤”专业相同的学号和姓名。

本例的查询任务实际上是同一学生表s的等值连接。

(1) 方法1

```
SELECT x.sno, x.sn  
FROM s AS x, s AS y  
WHERE x.maj = y.maj AND y.sn= '王彤';
```

(2) 方法2

```
SELECT x.sno, x.sn  
FROM s AS x INNER JOIN s AS y  
ON x.maj = y.maj AND y.sn= '王彤';
```

sno	sn
s1	王彤
s6	何欣荣

01

OPTION

单关系数据查询

02

OPTION

多关系数据查询

03

OPTION

子查询

04

OPTION

集合运算查询

1. 普通子查询

- ✓ 普通子查询的执行顺序是：首先执行子查询，然后把子查询的结果作为父查询的查询条件的值
- ✓ 普通子查询只执行一次，而父查询所涉及的所有记录行都与其查询结果进行比较以确定查询结果集合

1. 普通子查询

(1) 返回一个值的普通子查询

【例6-40】查询比学生“赵琳琳”年龄大的学号、姓名和年龄。

```
SELECT sno, sn, age
FROM s
WHERE age > (SELECT age
              FROM s
              WHERE sn='赵琳琳');
```

sno	sn	age
s2	苏乐	20
s6	何欣荣	21

本例的查询任务也可以用自连接查询实现，语句如下：

```
SELECT x.sno, x.sn, x.age
FROM s AS x, s AS y
WHERE x.age > y.age AND y.sn='赵琳琳';
```

1. 普通子查询

(1) 返回一个值的普通子查询

【例6-41】查询与教师“顾伟”职称不同的教师号、姓名和职称。

```
SELECT tno, tn, prof
FROM t
WHERE prof <> (SELECT prof
                FROM t
                WHERE tn='顾伟');
```

tno	tn	prof
t1	刘杨	教授
t2	石丽	讲师
t4	赵礼	教授
t6	张刚	讲师

本例的查询任务也可以用自连接查询实现，语句如下：

```
SELECT x.tno, x.tn, x.prof
FROM t AS x, t AS y
WHERE x.prof <> y.prof AND y.tn='顾伟';
```

1. 普通子查询

(1) 返回一个值的普通子查询

【例6-42】查询讲授“程序设计基础”课程的教师号、课程号和开课日期。

```
SELECT tno, cno, tcdte  
FROM tc  
WHERE cno = (SELECT cno  
             FROM c  
             WHERE cn='程序设计基础');
```

tno	cno	tcdte
t1	c2	2021-09-04
t3	c2	2021-03-08

本例的查询任务也可以用自连接查询实现，语句如下：

```
SELECT tno, tc.cno, tcdte  
FROM c, tc  
WHERE c.cno = tc.cno AND c.cn='程序设计基础';
```

1. 普通子查询

(2) 返回一组值的普通子查询

【例6-43】查询学号为“s2”的学生选修的课程号、课程名和课时。

```
SELECT cno, cn, ct
FROM c
WHERE cno = ANY (SELECT cno
                  FROM sc
                  WHERE sno='s2');
```

本例的查询任务也可以用自连接查询实现，语句如下：

```
SELECT c.cno, cn, ct
FROM c, sc
WHERE c.cno = sc.cno AND sno = 's2';
```

cno	cn	ct
c4	数据结构	64
c5	数据库系统	56
c6	数据挖掘	32
c7	高等数学	60

1. 普通子查询

(2) 返回一组值的普通子查询

【例6-44】查询其他专业中比“计算机”专业某一教师工资高的教师号、姓名、专业和工资。

```
SELECT tno, tn, maj, sal
FROM t
WHERE (sal > ANY (SELECT sal
                  FROM t
                  WHERE maj = '计算机'))
AND maj <> '计算机';
```

本例的查询任务也可以用如下语句实现：

```
SELECT tno, tn, maj, sal
FROM t
WHERE (sal > (SELECT MIN(sal)
              FROM t
              WHERE maj = '计算机'))
AND maj <> '计算机';
```

tno	tn	maj	sal
t4	赵礼	自动化	4267.90
t5	赵希希	数学	3332.67

1. 普通子查询

(2) 返回一组值的普通子查询

【例6-45】查询学号为“s2”的学生选修的课程号、课程名和课时（使用IN）。

```
SELECT cno, cn, ct
FROM c
WHERE cno IN (SELECT cno
               FROM sc
               WHERE sno='s2');
```

本例的查询任务和例6-43相同，例6-43中的父查询与子查询之间使用了“=ANY”，本例中使用“IN”代替“=ANY”，作用是相同的。

1. 普通子查询

(2) 返回一组值的普通子查询

【例6-46】查询选修课程号为“c1”的学号和姓名（使用IN）。

```
SELECT sno, sn
FROM s
WHERE sno IN (SELECT sno
              FROM sc
              WHERE cno = 'c1');
```

上述语句中的“IN”也可以用“=ANY”代替。

sno	sn
s1	王彤
s3	林欣怡
s4	陶然

1. 普通子查询

(2) 返回一组值的普通子查询

【例6-47】查询其他专业中比“计算机”专业所有教师工资高的教师号、姓名、专业和工资。

```
SELECT tno, tn, maj, sal
FROM t
WHERE (sal > ALL (SELECT sal
                  FROM t
                  WHERE maj = '计算机'))
AND maj <> '计算机';
```

本例的查询任务也可以用如下语句实现：

```
SELECT tno, tn, maj, sal
FROM t
WHERE (sal > (SELECT MAX(sal)
              FROM t
              WHERE maj = '计算机'))
AND maj <> '计算机';
```

tno	tn	maj	sal
t4	赵礼	自动化	4267.90

1. 普通子查询

(3) 用于数据操纵的普通查询

【例6-48】 求出各学院教师的平均工资，把结果存放在新表avgсал中。

首先，建立新表avgсал，用来存放学院名称和各个学院的平均工资。

```
use teaching;  
DROP TABLE IF EXISTS `avgсал`;  
CREATE TABLE avgсал  
(department VARCHAR(20),  
average SMALLINT);
```

然后，利用子查询求出教师表t中各学院的平均工资，把结果存放在新表avgсал中。

```
INSERT INTO avgсал  
SELECT dept, AVG(sal)  
FROM t  
GROUP BY dept;
```

执行上述语句之后，可以通过“SELECT * FROM avgсал;”查看新表中的数据。

1. 普通子查询

(3) 用于数据操纵的普通查询

【例6-49】把教师号为“t1”的教师讲授的课程的课时增加16学时。

```
UPDATE c
SET ct = ct + 16
WHERE cno IN (SELECT cno
              FROM tc
              WHERE tno = 't1');
```

子查询的作用是得到教师号为“t1”的教师讲授的课程号“c1”和“c2”。

执行上述语句之后，使用“SELECT * FROM c;”即可看出教师号为“t1”的教师讲授的课程的学时在原来学时的基础上增加了16学时。

1. 普通子查询

(3) 用于数据操纵的普通查询

【例6-50】把所有教师的工资提高到平均工资的1.2倍。

```
UPDATE t  
SET sal = (SELECT 1.2 * AVG(sal)  
           FROM t);
```

子查询的作用是得到所有教师的平均工资的1.2倍。

执行上述语句之后，使用“SELECT * FROM t;”即可看出所有教师的新工资情况。

2. 相关子查询

- ✓ 普通子查询，子查询中的查询条件不涉及父查询中的基本表的属性
- ✓ 有些查询任务中，子查询的查询条件需要引用父查询表中的属性值，这类查询称为相关子查询
- ✓ 相关子查询的执行顺序是：
 - 首先，选取父查询表中的第一行记录，子查询利用此行中相关的属性值在子查询设计的基本表中进行查询
 - 然后，父查询根据子查询返回的结果判断父查询表中的此行是否满足查询条件。如果满足条件，则把该行放入父查询的查询结果集合中
 - 重复执行这一过程，直到处理完父查询表中的每一行数据
- ✓ 由此可以看出，相关子查询的执行次数是由父查询表的行数决定的

2. 相关子查询

【例6-51】查询选修了课程号为“c1”的学号和姓名（使用相关子查询）。

本例的查询任务与例6-46相同，例6-46使用了普通子查询。本例中使用相关子查询，语句如下：

```
SELECT sno, sn  
FROM s  
WHERE 'c1' IN (SELECT cno  
                FROM sc  
                WHERE sno = s.sno);
```

上述语句中的“IN”的含义相当于子查询结果中的任何一个值，也可使用“=ANY”代替“IN”。

本例的相关子查询中，子查询的WHERE子句中用到了父查询表，即学生表s中的属性值。学生表s中的每一行即每个学生记录都要执行一次子查询，以确定该学生是否选修课程号为“c1”的课程，当“c1”课程是学生选修的课程时，该学生会被选取到父查询的查询结果中。

2. 相关子查询

【例6-52】查询没有选修课程号为“c1”的学号和姓名（使用相关子查询）。

```
SELECT sno, sn
FROM s
WHERE 'c1' NOT IN (SELECT cno
                   FROM sc
                   WHERE sno = s.sno);
```

本例也可以使用普通子查询实现，语句如下：

```
SELECT sno, sn
FROM s
WHERE sno NOT IN (SELECT sno
                  FROM sc
                  WHERE cno = 'c1');
```

sno	sn
s2	苏乐
s5	魏立
s6	何欣荣
s7	赵琳琳
s8	李轩

2. 相关子查询

- ✓ 使用EXISTS也可以进行相关子查询
- ✓ EXISTS是表示存在的量词，带有EXISTS的子查询不返回任何实际数据，它只得到逻辑值“真”或“假”
- ✓ 当子查询的查询结果集合为非空时，外层的WHERE子句返回真值，否则返回假值。NOT EXISTS与此相反。

【例6-53】查询选修了课程号为“c1”的学号和姓名（使用EXISTS）。

```
SELECT sno, sn  
FROM s  
WHERE EXISTS (SELECT *  
               FROM sc  
               WHERE sno = s.sno AND cno = 'c1');
```

2. 相关子查询

【例6-54】查询没有选修课程号为“c1”的学号和姓名（使用EXISTS）。

```
SELECT sno, sn  
FROM s  
WHERE NOT EXISTS (SELECT *  
                   FROM sc  
                   WHERE sno = s.sno AND cno = 'c1');
```

【例6-55】查询教师号为“t2”的教师讲授的课程号、课程名和课时（使用EXISTS）。

```
SELECT cno, cn, ct  
FROM c  
WHERE EXISTS (SELECT *  
              FROM tc  
              WHERE cno = c.cno AND tno = 't2');
```

01

OPTION

单关系数据查询

02

OPTION

多关系数据查询

03

OPTION

子查询

04

OPTION

集合运算查询

- ✓ 集合运算查询是使用UNION关键字将来自不同查询的数据组合起来，形成一个具有综合信息的查询结果
- ✓ UNION操作会自动将重复的数据行剔除
- ✓ 参加合并查询的各子查询的使用的表结构应该相同，即各子查询中的数据属性的数目和对应的数据类型都必须相同

【例6-56】查询“计算机”专业的学生信息，再查询“数学”专业的学生信息，将两个查询结果合并成一个结果集。

```
SELECT *  
FROM s  
WHERE maj = '计算机'  
UNION  
SELECT *  
FROM s  
WHERE maj = '数学';
```

sno	sn	sex	age	maj	dept
s1	王彤	女	18	计算机	信息学院
s6	何欣荣	女	21	计算机	信息学院
s5	魏立	男	17	数学	理学院
s7	赵琳琳	女	19	数学	理学院

【例6-57】查询课程号为“c1”的课程的总分和平均分，再查询课程号为“c2”的课程的总分和平均分，将两个查询结果合并成一个结果集。

```
SELECT cno AS 课程号, SUM(score) AS 总分, AVG(score) AS 平均分
FROM sc
WHERE cno = 'c1'
GROUP BY cno
UNION
SELECT cno AS 课程号, SUM(score) AS 总分, AVG(score) AS 平均分
FROM sc
WHERE cno = 'c2';
GROUP BY cno;
```

课程号	总分	平均分
c1	258.50	86.166667
c2	391.50	78.300000



人邮教育

www.ryjiaoyu.com

更多样书申请和资源下载需求，请登录人邮教育社区
(www.ryjiaoyu.com)

海量图书方便查询

囊括各大品类，您想要的应有尽有

免费申请样书

教师免费申请样书，
我们将安排快递迅速送达

下载配套资源

教学视频、PPT课件、教学案例、
习题答案、模拟试卷等丰富资源
免费下载



优惠购书

教师可以申请最低折扣
学生直接优惠购买图书

成为作者

欢迎写文章 / 投稿，我们强大的编辑团队将为您提供专业和高效的编辑出版服务